

SVM - Implementation

Chen Hajaj



Breast Cancer Classification Using Support Vector Machine (SVM)

Breast cancer

- ▶ **Breast cancer** is the most common cancer amongst women in the world. It accounts for 25% of all cancer cases and affected over 2.1 Million people in 2015 alone.
- ▶ Early diagnosis significantly increases the chances of survival. The key challenges against its detection is how to classify tumors into malignant (cancerous) or benign (noncancerous).
- ▶ Machine Learning technique can dramatically improve the level of diagnosis in breast cancer.
- ▶ Research shows that experienced physicians can detect cancer by 79% accuracy, while a 91 %(sometimes up to 97%) accuracy can be achieved using Machine Learning techniques.

Goal

- ▶ Classify tumors into malignant (cancerous) or benign (non-cancerous) using features obtained from several cell images.
- ▶ Features are computed from a digitized image of a fine needle aspirate (FNA) of a breast mass.

Attribute Information:

- ▶ ID number
- ▶ Diagnosis (M = malignant, B = benign)

Ten real-valued features are computed for each cell nucleus:

- Radius (mean of distances from center to points on the perimeter)
- Texture (standard deviation of gray-scale values)
- Perimeter
- Area
- Smoothness (local variation in radius lengths)
- Compactness ($\text{perimeter}^2 / \text{area} - 1.0$)
- Concavity (severity of concave portions of the contour)
- Concave points (number of concave portions of the contour)
- Symmetry
- Fractal dimension (“coastline approximation” – 1)

Loading Python Libraries and Breast Cancer Dataset

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

#Import Cancer data from the Sklearn library
# Dataset can also be found here (http://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+%28diagnostic%29)

from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()
```

Analyzing the data

```
In [3]: df_cancer = pd.DataFrame(np.c_[cancer['data'], cancer['target']], columns = np.append(cancer['feature_names'], ['target']),
df_cancer.head()
```

Out[3]:

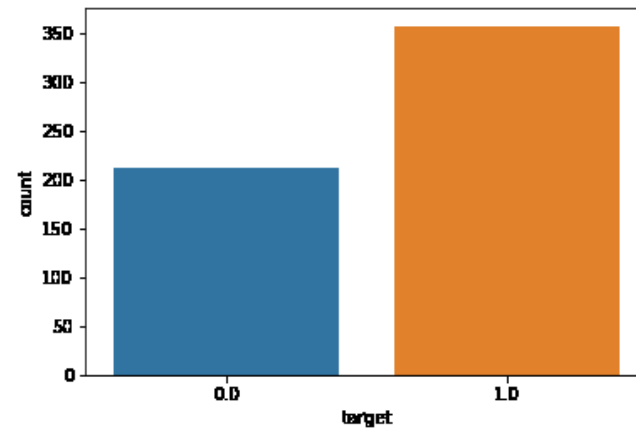
	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst texture	worst perimeter	worst area	worst smoothness	worst compactness
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	17.33	184.60	2019.0	0.1622	0.05468
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	23.41	158.80	1956.0	0.1238	0.07678
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	25.53	152.50	1709.0	0.1444	0.08919
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	26.50	98.87	567.7	0.2098	0.05711
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	16.67	152.20	1575.0	0.1374	0.07693

5 rows x 31 columns

we have 596 rows (Instances) and 31 columns (Features)

How many Benign and Malignant do we have in our dataset?

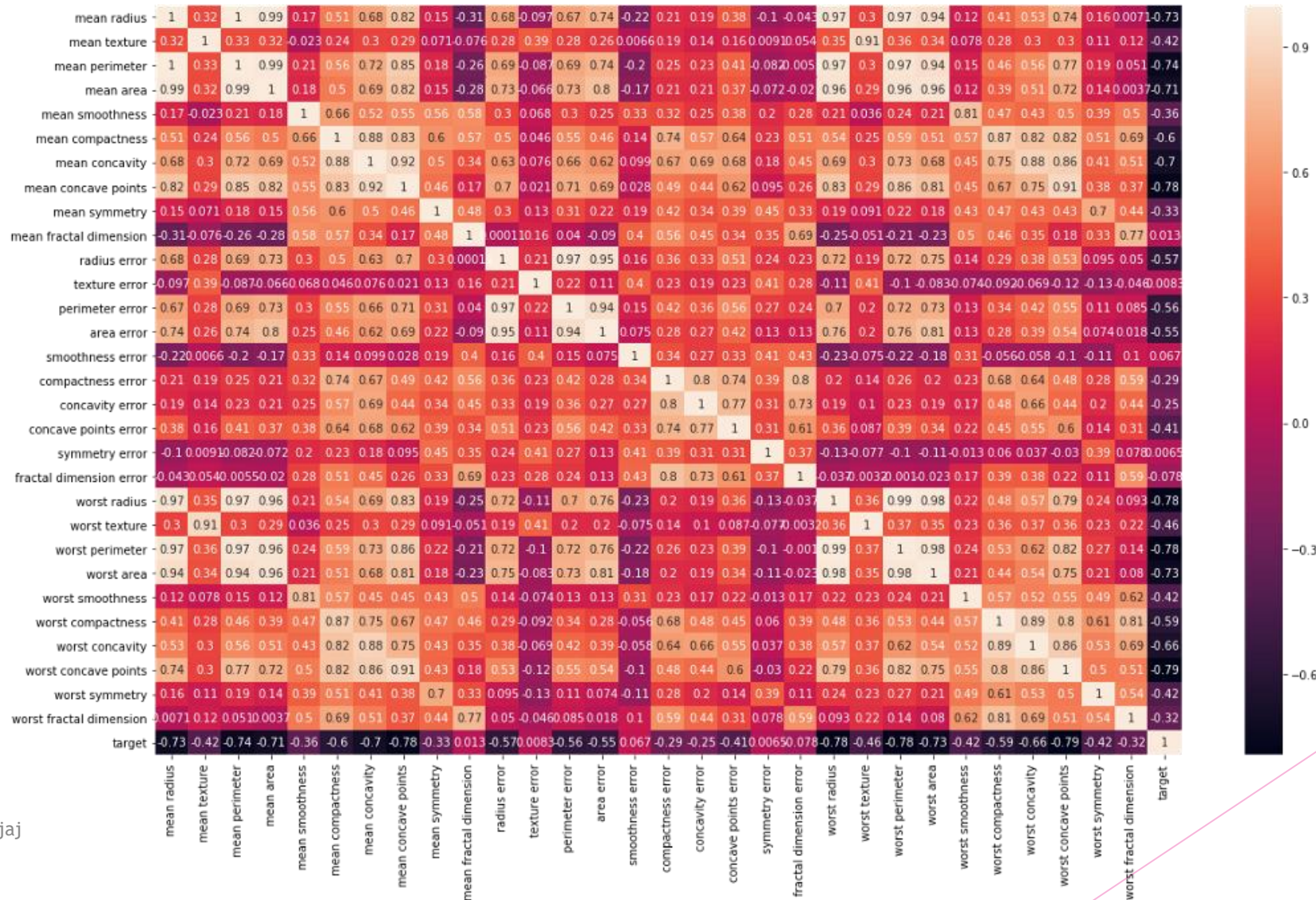
► `sns.countplot(df_cancer['target'], label = "Count")`



1.0	357
0.0	212


```
In [10]: plt.figure(figsize=(20,12))
sns.heatmap(df_cancer.corr(), annot=True)
```

Correlation between features



Modeling

```
X = df_cancer.drop(['target'], axis = 1)
# We drop our "target" feature and use all the remaining features in our dataframe
to train the model.

y = df_cancer['target']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
random_state = 20)

from sklearn.svm import SVC

svc_model = SVC()

svc_model.fit(X_train, y_train)
```

Prediction

```
y_predict = svc_model.predict(X_test)
```

- ▶ A confusion matrix for a binary classification task:

	Predicted Negative	Predicted Positive
Actual Negative	True Negative (TN)	False Positive (FP)
Actual Positive	False Negative (FN)	True Positive (TP)

- ▶ True Positive: A positive class observation (1) is correctly classified as positive by the model.
- ▶ False Positive: A negative class observation (0) is incorrectly classified as positive.
- ▶ True Negative: A negative class observation is correctly classified as negative.
- ▶ False Negative: A positive class observation is incorrectly classified as negative.

Prediction

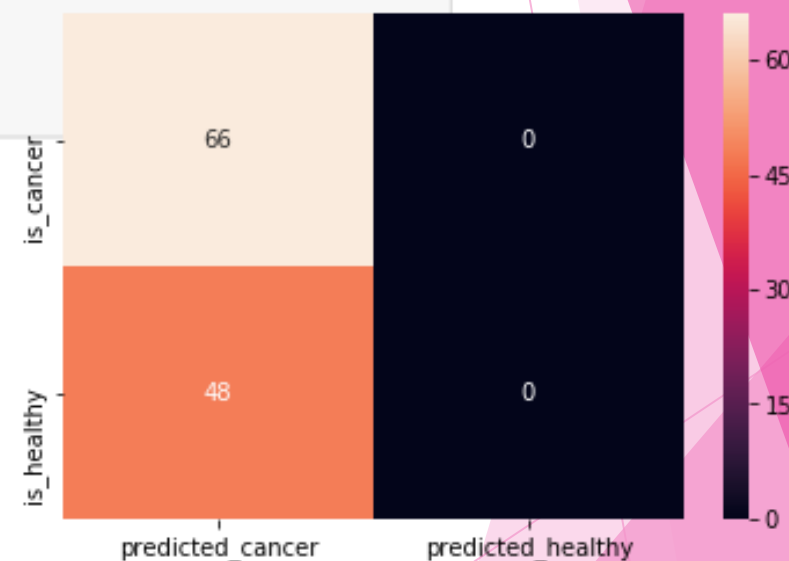
In [20]: *# Import metric libraries*

```
from sklearn.metrics import classification_report, confusion_matrix
```

```
In [21]: cm = np.array(confusion_matrix(y_test, y_predict, labels=[1,0]))
confusion = pd.DataFrame(cm, index=['is_cancer', 'is_healthy'],
                          columns=['predicted_cancer', 'predicted_healthy'])
confusion
```

Out[21]:

	predicted_cancer	predicted_healthy
is_cancer	66	0
is_healthy	48	0



```
In [22]: sns.heatmap(confusion, annot=True)
```

OOPS

```
In [23]: print(classification_report(y_test, y_predict))
```

	precision	recall	f1-score	support
0.0	0.00	0.00	0.00	48
1.0	0.58	1.00	0.73	66
avg / total	0.34	0.58	0.42	114

- ▶ Our model did not do a good job in its predictions. It predicted that 48 healthy patients have cancer.
- ▶ We only achieved 34% accuracy!

Improving our Model

► Normalization

```
In [27]: X_train_scaled = (X_train - X_train_min)/(X_train_range)
X_train_scaled.head()
```

Out[27]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area
412	0.114345	0.391003	0.110290	0.053150	0.293907	0.126219	0.087512	0.025487	0.108081	0.401860	...	0.072394	0.418354	0.080681	0.0284
461	0.967343	0.549827	0.988943	1.000000	0.605735	0.550334	0.851687	0.839463	0.505556	0.145814	...	1.000000	0.509582	1.000000	1.0000
532	0.317052	0.205882	0.303849	0.183245	0.435973	0.163088	0.041050	0.093439	0.288384	0.269535	...	0.281750	0.208097	0.254943	0.1445
495	0.373373	0.340138	0.361620	0.227953	0.469643	0.196522	0.159888	0.246074	0.215657	0.174884	...	0.287442	0.431579	0.266398	0.1470
13	0.419755	0.469550	0.414000	0.271135	0.340828	0.247899	0.232849	0.266600	0.397475	0.079535	...	0.316969	0.409447	0.306738	0.1695

5 rows × 30 columns

```
In [28]: X_test_min = X_test.min()
X_test_range = (X_test - X_test_min).max()
X_test_scaled = (X_test - X_test_min)/X_test_range
```

Chen Hajaj

Train again

```
In [29]: svc_model = SVC()  
svc_model.fit(X_train_scaled, y_train)
```

```
Out[29]: SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,  
decision_function_shape='ovr', degree=3, gamma='auto', kernel='rbf',  
max_iter=-1, probability=False, random_state=None, shrinking=True,  
tol=0.001, verbose=False)
```

```
In [30]: y_predict = svc_model.predict(X_test_scaled)  
cm = confusion_matrix(y_test, y_predict)
```

```
In [31]: cm = np.array(confusion_matrix(y_test, y_predict, labels=[1,0]))  
confusion = pd.DataFrame(cm, index=['is_cancer', 'is_healthy'],  
                          columns=['predicted_cancer', 'predicted_healthy'])  
confusion
```

Out[31]:

	predicted_cancer	predicted_healthy
is_cancer	66	0
is_healthy	1	47

Our prediction got a lot better with only 1 false prediction(Predicted cancer instead of healthy). We achieved 98% accuracy!

